

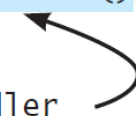
28 Lambda, Stream and Run-Time Type Identification(RTTI)

Inner Class Listeners

A listener class is designed specifically to create a listener object for a GUI component (e.g., a button). It will not be shared by other applications. So, **it is appropriate to define the listener class inside the frame class as an inner class.**

Anonymous Inner Classes

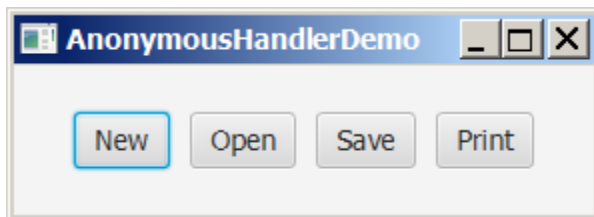
```
public void start(Stage primaryStage) {  
    // Omitted  
  
    btEnlarge.setOnAction(  
        new EnlargeHandler());  
}  
  
class EnlargeHandler  
    implements EventHandler<ActionEvent> {  
    public void handle(ActionEvent e) {  
        circlePane.enlarge();  
    }  
}
```



(a) Inner class EnlargeListener

```
public void start(Stage primaryStage) {  
    // Omitted  
  
    btEnlarge.setOnAction(  
        new class EnlargeHandler  
            implements EventHandler<ActionEvent>() {  
                public void handle(ActionEvent e) {  
                    circlePane.enlarge();  
                }  
            }  
    );  
}
```

(b) Anonymous inner class



[AnonymousHandlerDemo](#)

Run

Simplifying Event Handling Using Lambda Expressions

Lambda expression is a new feature in Java 8. Lambda expressions can be viewed as an **anonymous method with a concise syntax**. For example, the following code in (a) can be greatly simplified using a lambda expression in (b) in three lines.

```
btEnlarge.setOnAction(  
    new EventHandler<ActionEvent>() {  
        @Override  
        public void handle(ActionEvent e) {  
            // Code for processing event e  
        }  
    }  
);
```

(a) Anonymous inner class event handler

```
btEnlarge.setOnAction(e -> {  
    // Code for processing event e  
});
```

(b) Lambda expression event handler

Basic Syntax for a Lambda Expression

The basic syntax for a lambda expression is either

(type1 param1, type2 param2, ...) -> expression

or

(type1 param1, type2 param2, ...) -> { statements; }

The data type for a parameter may be explicitly declared or implicitly **inferred** by the compiler.

The parentheses **can be omitted** if there is only one parameter without an explicit data type.

Basic Syntax for a Lambda Expression

- A lambda consists of a parameter list followed by the arrow token and a body, as in:
 - *(parameterList) -> {statements}*
- For example, the following lambda receives two `ints` and returns their sum:
 - `(int x, int y) -> {return x + y;}`
- This lambda's body is a statement block that may contain one or more statements enclosed in curly braces.
- A lambda's **parameter types may be omitted**, as in:
 - `(x, y) -> {return x + y;}`
- in which case, the parameter and return types are determined by the lambda's context.

Basic Syntax for a Lambda Expression

- A lambda with a **one-expression body** can be written as:
 - `(x, y) -> x + y`
 - In this case, the expression's value is implicitly returned.
- When the parameter list contains only one parameter, **the parentheses may be omitted**, as in:
 - `value -> System.out.printf("%d ", value)`
- A lambda with an empty parameter list is defined with `()` to the left of the arrow token `->`, as in:
 - `() -> System.out.println("welcome to lambdas!")`
- There are also specialized shorthand forms of lambdas that are known as method references.

Sample Examples

```
1 //这里省略list的构造
2 List<String> names = ...;
3 Collections.sort(names, (o1, o2) -> o1.compareTo(o2));
```

```
1 //这里省略list的构造
2 List<String> names = ...;
3 Collections.sort(names, new Comparator<String>() {
4     @Override
5     public int compare(String o1, String o2) {
6         return o1.compareTo(o2);
7     }
8 });
```



为什么可以这样简化？会不会有简化的冲突？（如两个方法简化为一个Lambda表达式）

使用lambda表达式来排序和使用匿名内部类来排序。这个例子可以很明显的看出lambda表达式简化代码的效果。

怎么用Lambda表达式

- 任何可以接受一个FI(**Functional Interface, 函数式接口**)实例的地方，都可以用Lambda表达式。
- 比如，虽然上面给出的例子都是把Lambda表达式当作方法参数传递，可以接受FI作为参数的地方，都可以替换为Lambda表达式。
- Java8引入了函数式接口，在java.util.function包含了几大类函数式接口声明。

函数式接口

- 任何接口，如果只包含唯一一个抽象方法，那么它就是一个FI。（之前它们被称为SAM类型，即单抽象方法类型(Single Abstract Method)）。接口中的方法默认就是public abstract的。
- API作者们可以通过 @FunctionalInterface 注解来显式指定一个接口是函数式接口。加上这个注解，接口中函数式接口方法少于一个或者多余一个，编译器都会提示错误。

```
@FunctionalInterface
public interface Func {
    void run();
}
```

函数式接口，有如下特点：

- 有且只有一个抽象方法
- 可以有多个静态方法
- 可以有多个default方法(默认方法)
- 可以有多个Object的public的抽象方法

函数式接口

```
@FunctionalInterface
public interface Func {
}
```

编译错，编译器会告诉你*no target method*。

```
@FunctionalInterface
public interface Func {
    void run();
    void foo();
}
```

错误，只能一个接口

```
@FunctionalInterface
public interface Func {
    boolean equals(Object obj);
}
```

错误依旧，因为这个方法签名是Object类的public方法。

```
@FunctionalInterface
public interface Func {
    boolean equals(Object obj);
    void run();
}
```

OK，一个抽象方法，一个Object的public方法。

```
@FunctionalInterface
public interface Func {
    Object clone();
    void run();
}
```

不行，要Object的public方法，而clone是protected的。。

函数式接口，有且仅有一个抽象方法，Object的public方法除外。

函数式接口

一个普通接口NonFunc:

```
public interface NonFunc {  
    void foo();  
    void voo();  
}
```

函数式接口Func:

```
@FunctionalInterface  
public interface Func extends NonFunc {  
  
    void run();  
  
    default void foo() {  
        // do something  
    }  
  
    default void voo() {  
        // do something  
    }  
}
```

```
@FunctionalInterface  
interface GreetingService {  
  
    /**  
     * sayMessage  
     *  
     * @param message String  
     */  
    void sayMessage(String message);  
  
    /**  
     * 函数式接口里允许定义默认方法  
     */  
    default void doSomeMoreWork() {  
        // Method body  
    }  
  
    /**  
     * 函数式接口里允许定义静态方法  
     */  
    static void printHello() {  
        System.out.println("Hello");  
    }  
  
    /**  
     * 函数式接口里允许定义 java.lang.Object 里的 public 方法  
     *  
     * @param obj Object  
     * @return boolean  
     */  
    @Override  
    boolean equals(Object obj);  
}
```

Lambda与函数式接口

```
@FunctionalInterface
public interface Fitest {
    int add(int a, int b);
}
```

```
public class LamJava8 {
    public static void main(String[] args) {
        int a = 2, b = 3;
        int result = addMethod( a, b, (c, d) -> { return c+d; } );
        System.out.println( result );
    }

    public static int addMethod(int a, int b, Fitest fi) {
        return fi.add(a, b);
    }
}
```

相当于

```
new Fitest{
    int add(int c, int d){
        return c+d;
    }
}
```

方法addMethod(int a, int b, Fitest fi) 第三个参数是我们自定义的FI, 因此这里可以用(c, d) -> { return c+d; }表达式, c, d是Fitest 中 add() 方法的两个参数, { return c+d; } 是add 方法的方法体。

Lambda与函数式接口

- Java8里关于函数式接口的包是`java.util.function`，里面全部是函数式接口。
- 主要包含几大类：Function、Predicate、Supplier、Consumer和*Operator（没有Operator接口，只有类似BinaryOperator这样的接口）。
- Function: 接收参数，并返回结果，主要方法 `R apply(T t)`
- Consumer: 接收参数，无返回结果, 主要方法为 `void accept(T t)`
- Supplier: 不接收参数，但返回结构，主要方法为 `T get()`
- Predicate: 接收参数，返回boolean值，主要方法为 `boolean test(T t)`

Lambda与函数式接口

name	type	description
Consumer	Consumer< T >	接收T对象, 不返回值
Predicate	Predicate< T >	接收T对象并返回boolean
Function	Function< T, R >	接收T对象, 返回R对象
Supplier	Supplier< T >	提供T对象（例如工厂）, 不接收值
UnaryOperator	UnaryOperator< T >	接收T对象, 返回T对象
BiConsumer	BiConsumer<T, U>	接收T对象和U对象, 不返回值
BiPredicate	BiPredicate<T, U>	接收T对象和U对象, 返回boolean
BiFunction	BiFunction<T, U, R>	接收T对象和U对象, 返回R对象
BinaryOperator	BinaryOperator< T >	接收两个T对象, 返回T对象

Lambda与函数式接口

以Function为例:

@FunctionalInterface

public interface Function<T, R> {

Function

Function< T, R >

接收T对象, 返回R对象

```
1 //将Function对象应用到输入的参数上, 然后返回计算结果。
2 R apply(T t);
```

函数式接口

```
1 //返回一个先执行当前函数对象apply方法再执行after函数对象apply方法的函数对象。
2 default <V> Function<T, V> andThen(Function<? super R, ? extends V> after) {
3     Objects.requireNonNull(after);
4     return (T t) -> after.apply(apply(t));
5 }
```

输入参数t, 先执行apply(t), 得到的结果, 再作为after.apply的参数

```
1 //返回一个先执行before函数对象apply方法再执行当前函数对象apply方法的函数对象
2 default <V> Function<V, R> compose(Function<? super V, ? extends T> before) {
3     Objects.requireNonNull(before);
4     return (V v) -> apply(before.apply(v));
5 }
```

输入参数t, 先执行before.apply(t), 得到的结果, 再作为apply的参数

compose 和 andThen 的不同之处是函数执行的顺序不同。compose 函数先执行参数, 然后执行调用者, 而 andThen 先执行调用者, 然后再执行参数。

Lambda与函数式接口

```
1 public static void main(String[] args) {  
2     Function<Integer, Integer> name = e -> e * 2;  
3     Function<Integer, Integer> square = e -> e * e;  
4     int value = name.①andThen(②square).apply(3);  
5     System.out.println("andThen value=" + value);  
6     int value2 = name.②compose(①square).apply(3);  
7     System.out.println("compose value2=" + value2);  
8     //返回一个执行了apply()方法之后只会返回输入参数的函数  
9     Object identity = Function.identity().apply("huohuo");  
10    System.out.println(identity);  
11 }
```

```
1 andThen value=36  
2 compose value2=18  
3 huohuo
```

相当于 定义了一个实现了Function<Integer, Integer>的匿名类，然后 apply的内容为 e->e*2 然后new了一个该类的对象为name

```

01 public class FunctionDemo {
02
03     //API which accepts an implementation of
04
05     //Function interface
06
07     static void modifyTheValue(int valueToBeOperated, Function<Integer, Integer> function){
08
09         int newValue = function.apply(valueToBeOperated);
10
11         /*
12          * Do some operations using the new value.
13          */
14
15         System.out.println(newValue);
16
17     }
18
19 }

```

```

01 public static void main(String[] args) {
02
03     int incr = 20;  int myNumber = 10;
04
05     modifyTheValue(myNumber, val-> val + incr);
06
07     myNumber = 15;  modifyTheValue(myNumber, val-> val * 10);
08
09     modifyTheValue(myNumber, val-> val - 100);
10
11     modifyTheValue(myNumber, val-> "somestring".length() + val - 100);
12
13 }

```

1	30
2	
3	150
4	
5	-85
6	
7	-75

Lambda与函数式接口

```
public class TestJ8Predicate {  
  
    public static void main(String[] args) {  
        TestJ8Predicate testJ8Predicate = new TestJ8Predicate();  
        testJ8Predicate.printBigValue(10, val -> val > 5);  
        testJ8Predicate.printBigValueAnd(10, val -> val > 5);  
        testJ8Predicate.printBigValueAnd(6, val -> val > 5);  
  
        //binary predicate  
        BiPredicate<Integer, Long> biPredicate = (x, y) -> x > 9 && y < 100;  
        System.out.println(biPredicate.test(100, 50L));  
    }  
  
    public void printBigValue(int value, Predicate<Integer> predicate) {  
        if (predicate.test(value)) {  
            System.out.println(value);  
        }  
    }  
  
    public void printBigValueAnd(int value, Predicate<Integer> predicate) {  
        if (predicate.and(v -> v < 8).test(value)) {  
            System.out.println("value < 8 : " + value);  
        } else {  
            System.out.println("value should < 8 at least.");  
        }  
    }  
}
```

```

public class Java8Tester {
    public static void main(String args[]){
        List<Integer> list = Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9);
        System.out.println("输出所有数据:");

        // Predicate<Integer> predicate = n -> true
        // n 是一个参数传递到 Predicate 接口的 test 方法
        // n 如果存在则 test 方法返回 true

        // 传递参数 n
        eval(list, n->true);

        // Predicate<Integer> predicate1 = n -> n%2 == 0
        // n 是一个参数传递到 Predicate 接口的 test 方法
        // 如果 n%2 为 0 test 方法返回 true

        System.out.println("输出所有偶数:");
        eval(list, n-> n%2 == 0 );

        // Predicate<Integer> predicate2 = n -> n > 3
        // n 是一个参数传递到 Predicate 接口的 test 方法
        // 如果 n 大于 3 test 方法返回 true

        System.out.println("输出大于 3 的所有数字:");
        eval(list, n-> n > 3 );
    }

    public static void eval(List<Integer> list, Predicate<Integer> predicate) {
        for(Integer n: list) {

            if(predicate.test(n)) {
                System.out.print(n + " ");
            }
        }
    }
}

```

输出

```

1  输出所有数据:
2  1 2 3 4 5 6 7 8 9 输出所有偶数:
3  2 4 6 8 输出大于 3 的所有数字:
4  4 5 6 7 8 9

```

```

@FunctionalInterface
public interface Predicate<T> {
    boolean test(T t);
    ...
}

```

Stream

- Stream也是Java 8.0的新特性
- 大家可以把Stream当成一个高级版本的Iterator。
 - 原始版本的Iterator，用户只能一个一个的遍历元素并对其执行某些操作；
 - 高级版本的Stream，用户只要给出需要对其包含的元素执行什么操作，比如“过滤掉长度大于10的字符串”、“获取每个字符串的首字母”等，具体这些操作如何应用到每个元素上，就给Stream就好了！

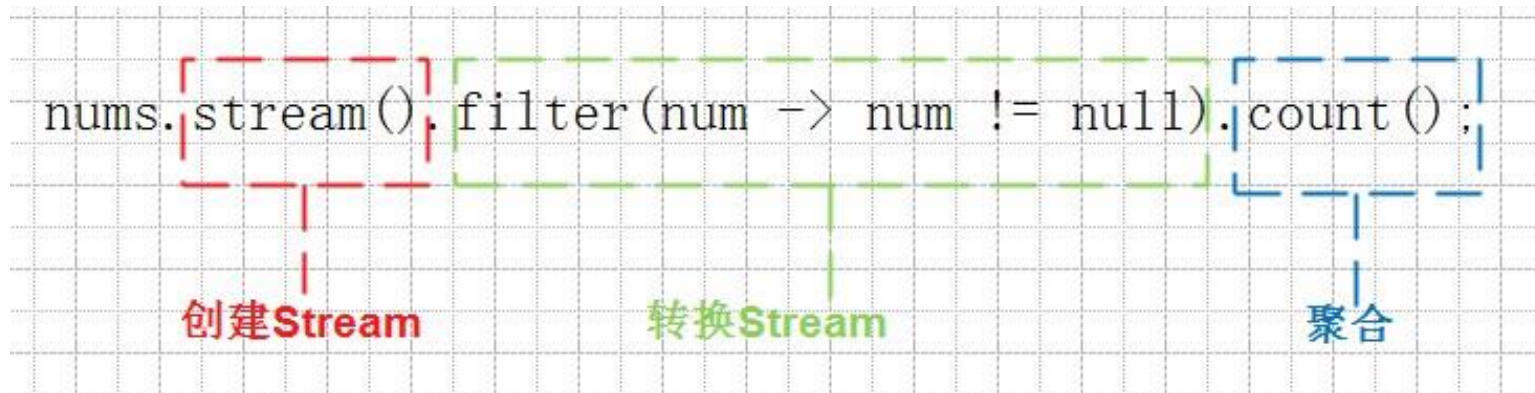
Stream

- Streams are objects that implement interface `Stream` (from the package `java.util.stream`)
 - Enable you to perform functional programming tasks
- Specialized stream interfaces for processing `int`, `long` or `double` values
- Streams move elements through a sequence of processing steps—known as a stream pipeline
 - Pipeline **begins** with a data source, **performs** various intermediate operations on the data source's elements and **ends** with a terminal operation.
- A stream pipeline is formed by chaining method calls.

An Example

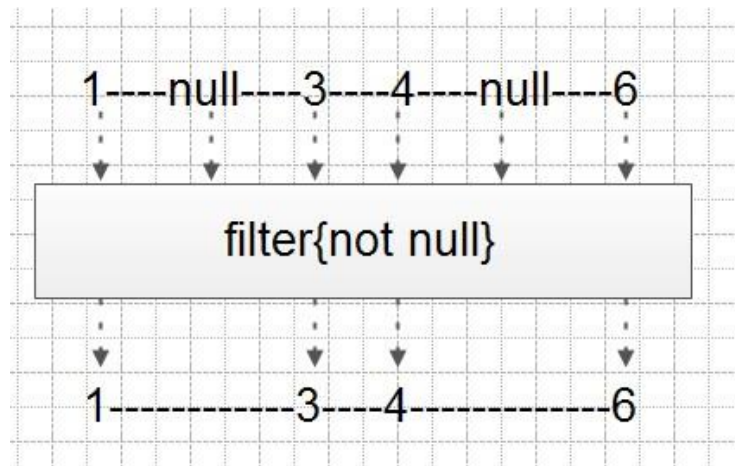
或用: `Arrays.asList`

```
1 //Lists是Guava中的一个工具类
2 List<Integer> nums = Lists.newArrayList(1,null,3,4,null,6);
3 nums.stream().filter(num -> num != null).count();
```



Stream使用基本步骤

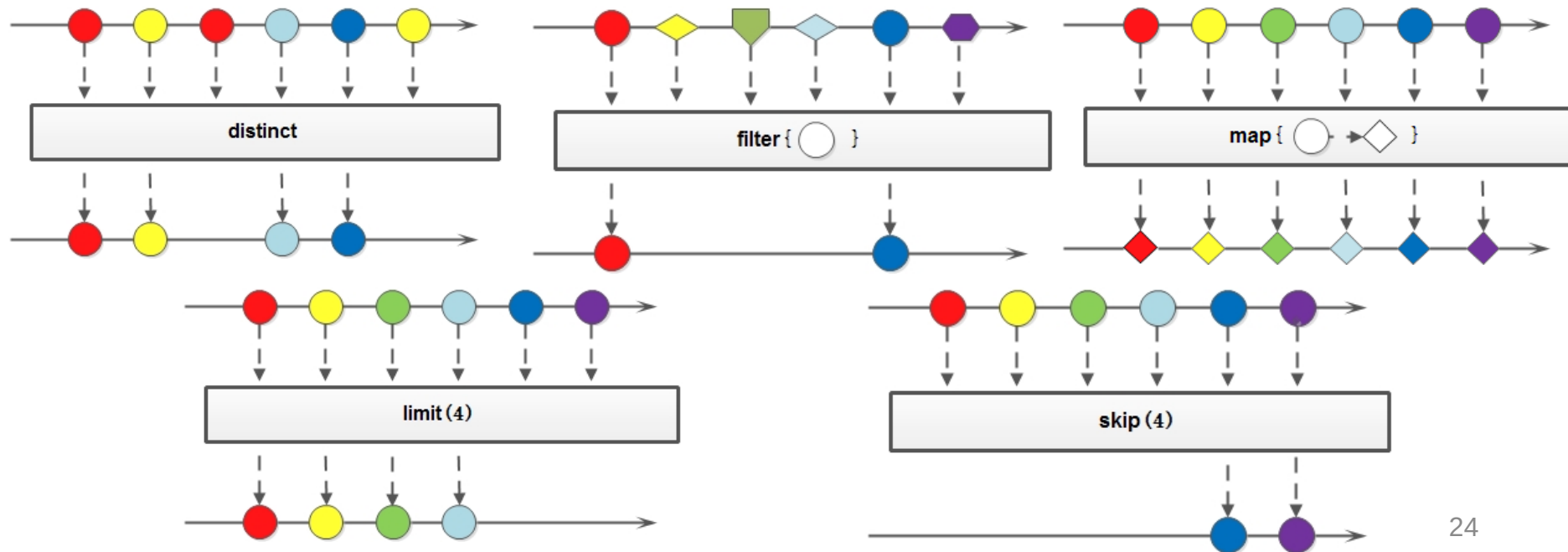
1. 创建Stream;
2. 转换Stream, 每次转换原有Stream对象不改变, 返回一个新的Stream对象 (**可以有**多次转换**);
3. 对Stream进行聚合 (Reduce) 操作, 获取想要的结果;



Intermediate Stream operations

filter	Results in a stream containing only the elements that satisfy a condition.
distinct	Results in a stream containing only the unique elements.
limit	Results in a stream with the specified number of elements from the beginning of the original stream.
map	Results in a stream in which <u>each element of the original stream is mapped to a new value (possibly of a different type)</u> —e.g., mapping numeric values to the squares of the numeric values. The new stream has the same number of elements as the original stream.
sorted	<u>Results in a stream in which the elements are in sorted order.</u> The new stream has the same number of elements as the original stream.

Fig. 17.3 | Common intermediate Stream operations.



forEach Performs processing on every element in a stream (e.g., display each element).

Reduction operations—*Take all values in the stream and return a single value*

average Calculates the *average* of the elements in a numeric stream.

count Returns the *number of elements* in the stream.

max Locates the *largest* value in a numeric stream.

min Locates the *smallest* value in a numeric stream.

reduce Reduces the elements of a collection to a *single value* using an associative accumulation function (e.g., a lambda that adds two elements).

汇聚操作（也称为规约操作）接受一个元素序列为输入，反复使用某个合并操作，把序列中的元素合并成一个汇总的结果。

Mutable reduction operations—*Create a container (such as a collection or `StringBuilder`)*

collect Creates a *new collection* of elements containing the results of the stream's prior operations.

toArray Creates an *array* containing the results of the stream's prior operations.

Search operations

findFirst Finds the *first* stream element based on the prior intermediate operations; immediately terminates processing of the stream pipeline once such an element is found.

findAny Finds *any* stream element based on the prior intermediate operations; immediately terminates processing of the stream pipeline once such an element is found.

anyMatch Determines whether *any* stream elements match a specified condition; immediately terminates processing of the stream pipeline if an element matches.

allMatch Determines whether *all* of the elements in the stream match a specified condition.

Fig. 17.4 | Common terminal Stream operations.

生成流

- 可以从各种数据源创建Stream，特别是Collections, List 和 Set, 集合接口有两个方法来生成流：
 - **stream()** – 为集合创建串行流。
 - **parallelStream()** – 为集合创建并行流。

```
List<String> strings = Arrays.asList("abc", "", "bc", "efg", "abcd","", "jkl");
```

```
List<String> filtered = strings.stream()  
                                .filter(string -> !string.isEmpty())  
                                .collect(Collectors.toList());
```

```
Arrays.asList("a1", "a2", "a3")  
    .stream()  
    .findFirst()  
    .ifPresent(System.out::println); // a1
```

Collection<E>

- ^A size() : int
- ^A isEmpty() : boolean
- ^A contains(Object) : boolean
- ^A iterator() : Iterator<E>
- ^A toArray() : Object[]
- ^A toArray(T[]) <T> : T[]
- ^A add(E) : boolean
- ^A remove(Object) : boolean
- ^A containsAll(Collection<?>) : boolean
- ^A addAll(Collection<? extends E>) : boolean
- ^A removeAll(Collection<?>) : boolean
- ^D removeIf(Predicate<? super E>) : boolean
- ^A retainAll(Collection<?>) : boolean
- ^A clear() : void
- ^A equals(Object) : boolean
- ^A hashCode() : int
- ^D splitter() : Splitter<E>
- ^D stream() : Stream<E>
- ^D parallelStream() : Stream<E>

```
* @return a sequential {@code Stream} over the elements in t.
* @since 1.8
*/
default Stream<E> stream() {
    return StreamSupport.stream(splitter(), false);
}

/**
 * Returns a possibly parallel {@code Stream} with this colle
 * source. It is allowable for this method to return a seque
 *
 * <p>This method should be overridden when the {@link #split
 * method cannot return a spliterator that is {@code IMMUTABLE
 * {@code CONCURRENT}, or <em>late-binding</em>. (See {@link
 * for details.)
 *
 * @implSpec
 * The default implementation creates a parallel {@code Strea
 * collection's {@code Spliterator}.
 *
 * @return a possibly parallel {@code Stream} over the elemen
 * collection
 * @since 1.8
 */
default Stream<E> parallelStream() {
    return StreamSupport.stream(splitter(), true);
}
```

- 也可以不创建集合来处理Stream，仅需要使用 **Stream.of()** 从一堆对象引用中创建一个Stream。
- 如下例所示：

```
Stream.of("a1", "a2", "a3")  
    .findFirst()  
    .ifPresent(System.out::println); // a1
```

- 除了常规的对象Stream，Java 8有特殊类型的Stream，用于处理基本数据类型int,long和double。它是IntStream、LongStream和DoubleStream。
- 如IntStreams可以使用IntStream.range()来代替常规的for循环。

```
IntStream.range(1, 4)  
    .forEach(System.out::println);  
  
// 1  
// 2  
// 3
```

由数组创建:

- Java8 中的 Arrays 的静态方法 stream() 可以获取数组流:
- static <T> Stream<T> stream(T[] array): 返回一个流
- 重载形式, 能够处理对应基本类型的数组:
 - public static IntStream stream(int[] array)
 - public static LongStream stream(long[] array)
 - public static DoubleStream stream(double[] array)

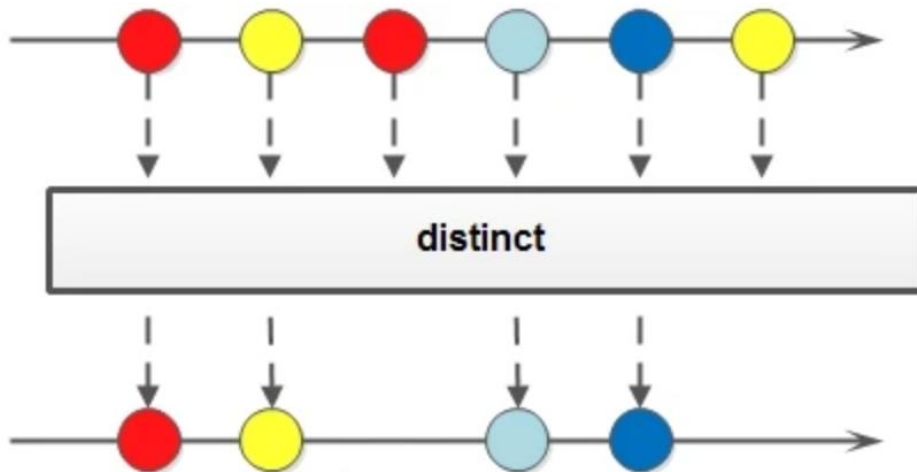
```
int[] intArray = {1,2,3};  
IntStream stream = Arrays.stream(intArray);
```

Stream的中间操作

- 如果Stream只有中间操作是不会执行的，当执行终端操作的时候才会执行中间操作，这种方式称为延迟加载或惰性求值。
- 多个中间操作组成一个中间操作链，只有当执行终端操作的时候才会执行一遍中间操作链。

Stream<T> distinct():

去重，通过流所生成元素的 hashCode() 和 equals() 去除重复元素。



```

public class StreamDemo {
    public static void main(String[] args) {
        //三个年轻人A、B、C报名了一次活动
        List<User> users = Lists.newArrayList();
        users.add(new User(15, "A"));
        users.add(new User(25, "B"));
        users.add(new User(21, "C"));
        users.add(new User(15, "A"));
        //由于A多报名了一次，现在活动方需要去重
        users.stream()
            .distinct()
            .forEach(user -> System.out.println(user.getName()));
        //输出: A、B、C
    }
}

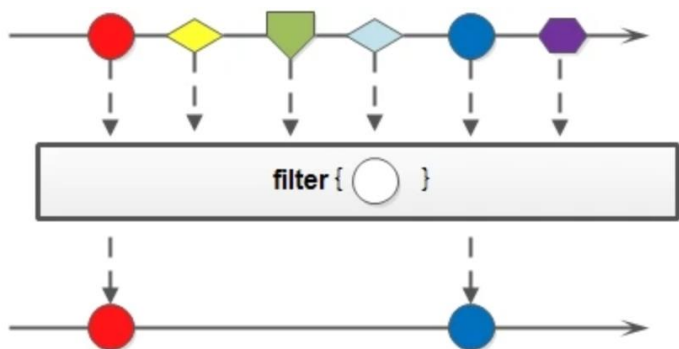
@NoArgsConstructor
@Data
class User {
    int age;
    String name;

    public User(int age, String name) {
        this.age = age;
        this.name = name;
    }
}

```

- Stream<T> filter(Predicate<? super T> predicate):

Predicate函数是断言型接口，filter方法中是接收一个和Predicate函数对应Lambda表达式，返回一个布尔值，从流中过滤某些元素。



```
public class StreamDemo {
    public static void main(String[] args) {
        //三个年轻人A、B、C报名了一次活动
        List<User> users = Lists.newArrayList();
        users.add(new User(15, "A"));
        users.add(new User(25, "B"));
        users.add(new User(21, "C"));
        //活动方规定大于15岁才能参加活动
        users.stream()
            .filter(user -> user.getAge() > 15)
            .forEach(user -> System.out.println(user.getName()));
        //输出: B、C
    }
}

@NoArgsConstructor
@Data
class User {
    int age;
    String name;

    public User(int age, String name) {
        this.age = age;
        this.name = name;
    }
}
```


- Stream<T> sorted(Comparator<? super T> comparator):
指定比较规则进行排序

```
public class StreamDemo {
    public static void main(String[] args) {
        //三个年轻人A、B、C报名了一次活动
        List<User> users = Lists.newArrayList();
        users.add(new User(15, "A"));
        users.add(new User(25, "B"));
        users.add(new User(21, "C"));
        //按照年龄进行排序
        users.stream()
            .sorted(Comparator.comparing(User::getAge))
            .forEach(user -> System.out.println(user.getName()));
        //输出: A、C、B
    }
}
```

user->user.getAge()

当使用 类名::实例方法名
方法引用时，一定是
lambda表达式所接收的第
一个参数来调用实例方法

@NoArgsConstructor

@Data

```
class User {
    int age;
    String name;

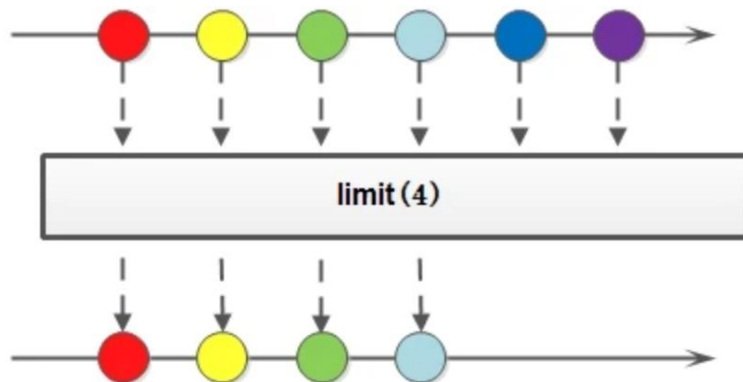
    public User(int age, String name) {
        this.age = age;
        this.name = name;
    }
}
```

```
Comparator<T>
    ^ compare(T, T) : int
    ^ equals(Object) : boolean
    ^ reversed() : Comparator<T>
    ^ thenComparing(Comparator<? super T>) : Comparator<T>
    ^ thenComparing(Function<? super T, ? extends U>, Comparator<? super U>) <U> : Comparator<T>
    ^ thenComparing(Function<? super T, ? extends U>) <U extends Comparable<? super U>> : Compar
    ^ thenComparingInt(ToIntFunction<? super T>) : Comparator<T>
    ^ thenComparingLong(ToLongFunction<? super T>) : Comparator<T>
    ^ thenComparingDouble(ToDoubleFunction<? super T>) : Comparator<T>
    ^ reverseOrder() <T extends Comparable<? super T>> : Comparator<T>
    ^ naturalOrder() <T extends Comparable<? super T>> : Comparator<T>
    ^ nullsFirst(Comparator<? super T>) <T> : Comparator<T>
    ^ nullsLast(Comparator<? super T>) <T> : Comparator<T>
    ^ comparing(Function<? super T, ? extends U>, Comparator<? super U>) <T, U> : Comparator<T>
    ^ comparing(Function<? super T, ? extends U>) <T, U extends Comparable<? super U>> : Comparator
    ^ comparingInt(ToIntFunction<? super T>) <T> : Comparator<T>
    ^ comparingLong(ToLongFunction<? super T>) <T> : Comparator<T>
    ^ comparingDouble(ToDoubleFunction<? super T>) <T> : Comparator<T>
```

```
public static <T, U extends Comparable<? super U>> Comparator<T> comparing(
    Function<? super T, ? extends U> keyExtractor)
{
    Objects.requireNonNull(keyExtractor);
    return (Comparator<T> & Serializable)
        (c1, c2) -> keyExtractor.apply(c1).compareTo(keyExtractor.apply(c2));
}
```

- `Stream<T> limit(long maxSize):`

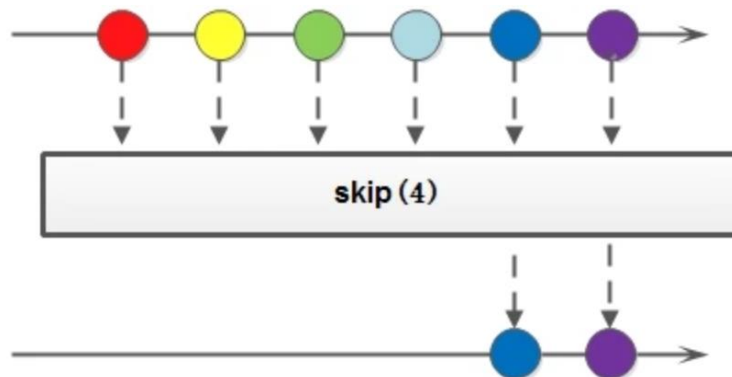
截断流，使其元素不超过给定数量。如果元素的个数小于maxSize，那就获取所有元素。



```
public class StreamDemo {
    public static void main(String[] args) {
        //三个年轻人A、B、C报名了一次活动
        List<User> users = Lists.newArrayList();
        users.add(new User(15, "A"));
        users.add(new User(25, "B"));
        users.add(new User(21, "C"));
        //按照年龄排序,倒排需要用到reversed方法,然后用limit取出第一个,看谁的年龄最大
        users.stream()
            .sorted(Comparator.comparing(User::getAge).reversed())
            .limit(1)
            .forEach(user -> System.out.println(user.getName()));
        //输出: B
    }
}
```

- `Stream<T> skip(long n):`

跳过元素，返回一个扔掉了前 n 个元素的流。若流中元素不足 n 个，则返回一个空流。



```
public class StreamDemo {  
    public static void main(String[] args) {  
        //三个年轻人A、B、C报名了一次活动  
        List<User> users = Lists.newArrayList();  
        users.add(new User(15, "A"));  
        users.add(new User(25, "B"));  
        users.add(new User(21, "C"));  
        //按照年龄排序,倒排需要用到reversed方法,然后用skip方法跳过前边年龄大的,看谁是年龄最小的  
        users.stream()  
            .sorted(Comparator.comparing(User::getAge).reversed())  
            .skip(users.size()-1)  
            .forEach(user -> System.out.println(user.getName()));  
        //输出: A  
    }  
}
```

- `Stream<R> map(Function<? super T, ? extends R> mapper)`:
接收一个Function函数作为参数，该函数会被应用到每个元素上，并将其映射成一个新的元素。

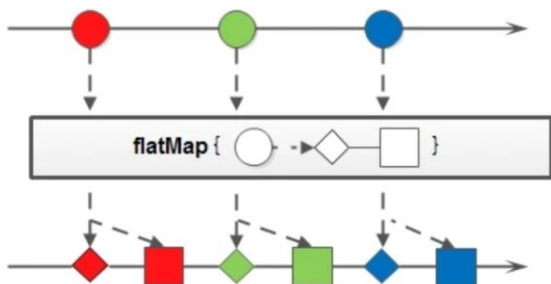
map还有三个应用于具体类型方法，分别是：`mapToInt`，`mapToLong`和`mapToDouble`。`mapToInt`就是把原始Stream转换成一个新的Stream，这个新生成的Stream中的元素都是int类型。这三个方法可以免除自动装箱/拆箱的额外消耗。

```
public class StreamDemo {  
    public static void main(String[] args) {  
        //三个年轻人A、B、C报名了一次活动  
        List<User> users = Lists.newArrayList();  
        users.add(new User(15, "A"));  
        users.add(new User(25, "B"));  
        users.add(new User(21, "C"));  
        //传入users，用map方法取出他们的名字并转换成欢迎语输出  
        users.stream()  
            .map(user -> "欢迎: " + user.getName())  
            .forEach(System.out::println);  
  
        /**  
         * 欢迎: A  
         * 欢迎: B  
         * 欢迎: C  
         */  
    }  
}
```

`.forEach(x->System.out.println(x));`



- `Stream<R> flatMap(Function<? super T, ? extends Stream<? extends R>> mapper)`:
- 接收一个Function函数作为参数，将流中的每个值都转换成另一个流，然后把所有流连接成一个流。



```
@NoArgsConstructor
@Data
class User {
    int age;
    String name;
    List<String> money;

    public User(int age, String name, List<String> money) {
        this.age = age;
        this.name = name;
        this.money = money;
    }
}
```

```
public class StreamDemo {
    public static void main(String[] args) {
        /**
         * 现在演示一下map和flatMap的场景，如果三个年轻人没每个人都带着若干钞票，
         * 那么我想看看他们每个人都带了多少怎么办？
         */
        List<User> users = Lists.newArrayList();
        users.add(new User(15, "A", ImmutableList.of("1元", "5元")));
        users.add(new User(25, "B", ImmutableList.of("10元", "50元")));
        users.add(new User(21, "C", ImmutableList.of("100元")));
        /**
         * map方式输出:
         * [1元, 5元]
         * [10元, 50元]
         * [100元]
         */
        users.stream()
            .map(user -> user.getMoney())
            .forEach(System.out::println);

        /**
         * flatMap方式输出:
         * 1元
         * 5元
         * 10元
         * 50元
         * 100元
         */
        users.stream()
            .flatMap(user -> user.getMoney().stream())
            .forEach(System.out::println);
        //对比两个方法的输出结果，map会将每个值返回一个流然后输出，flatMap会将每个值返回的流合并成一个流输出
    }
}
```

Stream的终端操作

终端操作执行中间操作链，并返回结果。

- `void forEach(Consumer<? super T> action):`

```
users.stream().forEach(user -> System.out.println(user.getName()));
```

- `<R, A> R collect(Collector<? super T, A, R> collector)`

```
List<User> users = Lists.newArrayList();
users.add(new User(15, "A", ImmutableList.of("1元", "5元")));
users.add(new User(25, "B", ImmutableList.of("10元", "50元")));
users.add(new User(21, "C", ImmutableList.of("100元")));
//收集名称到List
List<String> nameList = users.stream().map(User::getName).collect(Collectors.toList());
//收集名称到List
Set<String> nameSet = users.stream().map(User::getName).collect(Collectors.toSet());
//收集到map, 名字作为key, user对象作为value
Map<String, User> userMap = users.stream()
    .collect(Collectors.toMap(User::getName, Function.identity(), (k1, k2) -> k2));
```


• List转Map

```
Map<String, String> map = new HashMap<>();  
for (User user : userList) {  
    map.put(user.getId(), user.getName());  
}
```



```
userList.stream().collect(Collectors.toMap(User::getId, User::getName));
```

如果希望得到 Map 的 value 为对象本身时，可以这样写：

```
userList.stream().collect(Collectors.toMap(User::getId, t → t));  
或：  
userList.stream().collect(Collectors.toMap(User::getId, Function.identity()));
```

解决同Key的冲突： `user("A", "张三"), user("A", "李四"), user("C", "王五")`

```
userList.stream().collect(Collectors.toMap(User::getId, User::getName, (n1, n2) → n1 + n2));
```

// 输出结果：

A→ 张三李四

C→ 王五

•其他终端操作：

1.`boolean allMatch(Predicate<? super T> predicate)`; 检查是否匹配所有元素。

2.`boolean anyMatch(Predicate<? super T> predicate)`; 检查是否至少匹配一个元素。

3.`boolean noneMatch(Predicate<? super T> predicate)`; 检查是否没有匹配所有元素。

4.`Optional<T> findFirst()`; 返回当前流中的第一个元素。

5.`Optional<T> findAny()`; 返回当前流中的任意元素。

6.`long count()`; 返回流中元素总数。

7.`Optional<T> max(Comparator<? super T> comparator)`; 返回流中最大值。

8.`Optional<T> min(Comparator<? super T> comparator)`; 返回流中最小值。

9.`T reduce(T identity, BinaryOperator<T> accumulator)`; 可以将流中元素反复结合起来，得到一个值。返回 `T`。这是一个归约操作。

IntStream Operations

- **IntStream** (package **java.util.stream**)—a specialized stream for manipulating **int** values.
- The techniques shown in this example also apply to **LongStreams** and **DoubleStreams** for **long** and **double** values, respectively.

```
List<Integer> nums = Arrays.asList(1,2,3,4,5);  
nums.stream().filter.....  
也是可以的！
```

```

1 // Fig. 17.5: IntStreamOperations.java
2 // Demonstrating IntStream operations.
3 import java.util.Arrays;
4 import java.util.stream.IntStream;
5
6 public class IntStreamOperations
7 {
8     public static void main(String[] args)
9     {
10         int[] values = {3, 10, 6, 1, 4, 8, 2, 5, 9, 7};
11
12         // display original values
13         System.out.print("Original values: ");
14         IntStream.of(values)
15             .forEach(value -> System.out.printf("%d ", value));
16         System.out.println();
17
18         // count, min, max, sum and average of the values
19         System.out.printf("\nCount: %d\n", IntStream.of(values).count());
20         System.out.printf("Min: %d\n",
21             IntStream.of(values).min().getAsInt());
22         System.out.printf("Max: %d\n",
23             IntStream.of(values).max().getAsInt());
24         System.out.printf("Sum: %d\n", IntStream.of(values).sum());
25         System.out.printf("Average: %.2f\n",
26             IntStream.of(values).average().getAsDouble());
27
28         // sum of values with reduce method
29         System.out.printf("\nSum via reduce method: %d\n",
30             IntStream.of(values)
31                 .reduce(0, (x, y) -> x + y));
32
33         // sum of squares of values with reduce method
34         System.out.printf("Sum of squares via reduce method: %d\n",
35             IntStream.of(values)
36                 .reduce(0, (x, y) -> x + y * y));

```

```

37
38 // product of values with reduce method
39 System.out.printf("Product via reduce method: %d%n",
40     IntStream.of(values)
41         .reduce(1, (x, y) -> x * y));
42
43 // even values displayed in sorted order
44 System.out.printf("%nEven values displayed in sorted order: ");
45 IntStream.of(values)
46     .filter(value -> value % 2 == 0)
47     .sorted()
48     .forEach(value -> System.out.printf("%d ", value));
49 System.out.println();
50
51 // odd values multiplied by 10 and displayed in sorted order
52 System.out.printf(
53     "Odd values multiplied by 10 displayed in sorted order: ");
54 IntStream.of(values)
55     .filter(value -> value % 2 != 0)
56     .map(value -> value * 10)
57     .sorted()
58     .forEach(value -> System.out.printf("%d ", value));
59 System.out.println();
60
61 // sum range of integers from 1 to 10, exclusive
62 System.out.printf("%nSum of integers from 1 to 9: %d%n",
63     IntStream.range(1, 10).sum());
64
65 // sum range of integers from 1 to 10, inclusive
66 System.out.printf("Sum of integers from 1 to 10: %d%n",
67     IntStream.rangeClosed(1, 10).sum());
68 }
69 } // end class IntStreamOperations

```

Original values: 3 10 6 1 4 8 2 5 9 7

Count: 10

Min: 1

Max: 10

Sum: 55

Average: 5.50

Sum via reduce method: 55

Sum of squares via reduce method: 385

Product via reduce method: 3628800

Even values displayed in sorted order: 2 4 6 8 10

Odd values multiplied by 10 displayed in sorted order: 10 30 50 70 90

Sum of integers from 1 to 9: 45

Sum of integers from 1 to 10: 55

Stream<Integer> Manipulations

- Class **Array**'s **stream** method is used to create a **Stream** from an array of objects.

```
3 import java.util.Arrays;
4 import java.util.Comparator;
5 import java.util.List;
6 import java.util.stream.Collectors;
7
8 public class ArraysAndStreams
9 {
10     public static void main(String[] args)
11     {
12         Integer[] values = {2, 9, 5, 0, 3, 7, 1, 4, 8, 6};
13
14         // display original values
15         System.out.printf("Original values: %s\n", Arrays.asList(values));
16
17         // sort values in ascending order with streams
18         System.out.printf("Sorted values: %s\n",
19             Arrays.stream(values)
20                 .sorted()
21                 .collect(Collectors.toList()));
22     }
23 }
```

```

23 // values greater than 4
24 List<Integer> greaterThan4 =
25     Arrays.stream(values)
26         .filter(value -> value > 4)
27         .collect(Collectors.toList());
28 System.out.printf("Values greater than 4: %s\n", greaterThan4);
29
30 // filter values greater than 4 then sort the results
31 System.out.printf("Sorted values greater than 4: %s\n",
32     Arrays.stream(values)
33         .filter(value -> value > 4)
34         .sorted()
35         .collect(Collectors.toList()));
36
37 // greaterThan4 List sorted with streams
38 System.out.printf(
39     "Values greater than 4 (ascending with streams): %s\n",
40     greaterThan4.stream()
41         .sorted()
42         .collect(Collectors.toList()));
43 }
44 } // end class ArraysAndStreams

```

Original values: [2, 9, 5, 0, 3, 7, 1, 4, 8, 6]
 Sorted values: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
 Values greater than 4: [9, 5, 7, 8, 6]
 Sorted values greater than 4: [5, 6, 7, 8, 9]
 Values greater than 4 (ascending with streams): [5, 6, 7, 8, 9]

Stream<String> Manipulations

```
3 import java.util.Arrays;
4 import java.util.Comparator;
5 import java.util.stream.Collectors;
6
7 public class ArraysAndStreams2
8 {
9     public static void main(String[] args)
10    {
11        String[] strings =
12            {"Red", "orange", "Yellow", "green", "Blue", "indigo", "Violet"};
13
14        // display original strings
15        System.out.printf("Original strings: %s\n", Arrays.asList(strings));
16
17        // strings in uppercase
18        System.out.printf("strings in uppercase: %s\n",
19            Arrays.stream(strings)
20                .map(String::toUpperCase)
21                .collect(Collectors.toList()));
22        // strings less than "n" (case insensitive) sorted ascending
23        System.out.printf("strings greater than m sorted ascending: %s\n",
24            Arrays.stream(strings)
25                .filter(s -> s.compareToIgnoreCase("n") < 0)
26                .sorted(String.CASE_INSENSITIVE_ORDER)
27                .collect(Collectors.toList()));
28
29        // strings less than "n" (case insensitive) sorted descending
30        System.out.printf("strings greater than m sorted descending: %s\n",
31            Arrays.stream(strings)
32                .filter(s -> s.compareToIgnoreCase("n") < 0)
33                .sorted(String.CASE_INSENSITIVE_ORDER.reversed())
34                .collect(Collectors.toList()));
35    }
36 } // end class ArraysAndStreams2
```

Original strings: [Red, orange, Yellow, green, Blue, indigo, Violet]
strings in uppercase: [RED, ORANGE, YELLOW, GREEN, BLUE, INDIGO, VIOLET]
strings greater than m sorted ascending: [orange, Red, Violet, Yellow]
strings greater than m sorted descending: [Yellow, Violet, Red, orange]

Stream<Employee> Manipulations

```
1  // Fig. 17.9: Employee.java
2  // Employee class.
3  public class Employee
4  {
5      private String firstName;
6      private String lastName;
7      private double salary;
8      private String department;
9
10     // constructor
11     public Employee(String firstName, String lastName,
12                     double salary, String department)
13     {
14         this.firstName = firstName;
15         this.lastName = lastName;
16         this.salary = salary;
17         this.department = department;
18     }
19
20     // set firstName
21     public void setFirstName(String firstName)
22     {
23         this.firstName = firstName;
24     }
```



```
25
26 // get firstName
27 public String getFirstName()
28 {
29     return firstName;
30 }
31
32 // set lastName
33 public void setLastName(String lastName)
34 {
35     this.lastName = lastName;
36 }
37
38 // get lastName
39 public String getLastName()
40 {
41     return lastName;
42 }
43
44 // set salary
45 public void setSalary(double salary)
46 {
47     this.salary = salary;
48 }
```

```
49
50 // get salary
51 public double getSalary()
52 {
53     return salary;
54 }
55
56 // set department
57 public void setDepartment(String department)
58 {
59     this.department = department;
60 }
61
62 // get department
63 public String getDepartment()
64 {
65     return department;
66 }
67
68 // return Employee's first and last name combined
69 public String getName()
70 {
71     return String.format("%s %s", getFirstName(), getLastName());
72 }
```

```
73
74 // return a String containing the Employee's information
75 @Override
76 public String toString()
77 {
78     return String.format("%-8s %-8s %8.2f   %s",
79         getFirstName(), getLastName(), getSalary(), getDepartment());
80 } // end method toString
81 } // end class Employee
```

```

1 // Fig. 17.10: ProcessingEmployees.java
2 // Processing streams of Employee objects.
3 import java.util.Arrays;
4 import java.util.Comparator;
5 import java.util.List;
6 import java.util.Map;
7 import java.util.TreeMap;
8 import java.util.function.Function;
9 import java.util.function.Predicate;
10 import java.util.stream.Collectors;
11
12 public class ProcessingEmployees
13 {
14     public static void main(String[] args)
15     {
16         // initialize array of Employees
17         Employee[] employees = {
18             new Employee("Jason", "Red", 5000, "IT"),
19             new Employee("Ashley", "Green", 7600, "IT"),
20             new Employee("Matthew", "Indigo", 3587.5, "Sales"),
21             new Employee("James", "Indigo", 4700.77, "Marketing"),
22             new Employee("Luke", "Indigo", 6200, "IT"),
23             new Employee("Jason", "Blue", 3200, "Sales"),
24             new Employee("Wendy", "Brown", 4236.4, "Marketing")};
25
26         // get List view of the Employees
27         List<Employee> list = Arrays.asList(employees);
28
29         // display all Employees
30         System.out.println("Complete Employee list:");
31         list.stream().forEach(System.out::println);
32

```

Complete Employee list:

Jason	Red	5000.00	IT
Ashley	Green	7600.00	IT
Matthew	Indigo	3587.50	Sales
James	Indigo	4700.77	Marketing
Luke	Indigo	6200.00	IT
Jason	Blue	3200.00	Sales
Wendy	Brown	4236.40	Marketing

```

33 // Predicate that returns true for salaries in the range $4000-$6000
34 Predicate<Employee> fourToSixThousand =
35     e -> (e.getSalary() >= 4000 && e.getSalary() <= 6000);
36
37 // Display Employees with salaries in the range $4000-$6000
38 // sorted into ascending order by salary
39 System.out.printf(
40     "%nEmployees earning $4000-$6000 per month sorted by salary:%n");
41 list.stream()
42     .filter(fourToSixThousand)
43     .sorted(Comparator.comparing(Employee::getSalary))
44     .forEach(System.out::println);
45
46 // Display first Employee with salary in the range $4000-$6000
47 System.out.printf("%nFirst employee who earns $4000-$6000:%n%s%n",
48     list.stream()
49         .filter(fourToSixThousand)
50         .findFirst()
51         .get());
52

```

Employees earning \$4000-\$6000 per month sorted by salary:

Wendy	Brown	4236.40	Marketing
James	Indigo	4700.77	Marketing
Jason	Red	5000.00	IT

First employee who earns \$4000-\$6000:

Jason	Red	5000.00	IT
-------	-----	---------	----

```

91 // group Employees by department
92 System.out.printf("%nEmployees by department:%n");
93 Map<String, List<Employee>> groupedByDepartment =
94     list.stream()
95         .collect(Collectors.groupingBy(Employee::getDepartment));
96 groupedByDepartment.forEach(
97     (department, employeesInDepartment) ->
98     {
99         System.out.println(department);
100         employeesInDepartment.forEach(
101             employee -> System.out.printf("    %s%n", employee));
102     }
103 );
104

```

Employees by department:

Sales

Matthew	Indigo	3587.50	Sales
Jason	Blue	3200.00	Sales

IT

Jason	Red	5000.00	IT
Ashley	Green	7600.00	IT
Luke	Indigo	6200.00	IT

Marketing

James	Indigo	4700.77	Marketing
Wendy	Brown	4236.40	Marketing

```
105 // count number of Employees in each department
106 System.out.printf("%nCount of Employees by department:%n");
107 Map<String, Long> employeeCountByDepartment =
108     list.stream()
109         .collect(Collectors.groupingBy(Employee::getDepartment,
110             Collectors.counting()));
111 employeeCountByDepartment.forEach(
112     (department, count) -> System.out.printf(
113         "%s has %d employee(s)%n", department, count));
114
```

```
Count of Employees by department:
IT has 3 employee(s)
Marketing has 2 employee(s)
Sales has 2 employee(s)
```

- 输入:

```
Taylor Swift U.S.  
the Beatles U.K.  
the Dynasty Tang China  
the May Flower China
```

统计: 以"the"开始的行的国家

```
China  
U.K.
```

```
public class Main {  
    public static void main(String[] args) {  
        Scanner in = new Scanner(System.in);  
        ArrayList<String[]> lst = new ArrayList<String[]>();  
        while ( in.hasNextLine() ) {  
            String line = in.nextLine();  
            lst.add(line.split(" "));  
        }  
        Set<String> ret = lst.stream()  
            .filter([                ])  
            .map([                ])  
            .collect(Collectors.[ ]);  
        ArrayList<String> r = new ArrayList<String>(ret);  
        Collections.sort(r);  
        for ( String s: r ) {  
            System.out.println(s);  
        }  
        in.close();  
    }  
}
```

程序填空题

```
public class Main1 {  
    public static void main(String[] args) {  
        Scanner in = new Scanner(System.in);  
        ArrayList<String[]> lst = new ArrayList<String[]>();  
        while (in.hasNextLine()) {  
            String line = in.nextLine();  
            if(line.equals(""))  
                break;  
            lst.add(line.split(" "));  
        }  
        Set<String> ret = lst.stream().filter(x->x[0].equals("the")).map(x->x[x.length-1])  
            .collect(Collectors.toSet());  
        ArrayList<String> r = new ArrayList<String>(ret);  
        Collections.sort(r);  
        for (String s: r) {  
            System.out.println(s);  
        }  
        in.close();  
    }  
}
```

What is Reflection

- **Reflection:** *the process by which a program **can observe and modify its own structure and behavior at runtime**.*
- Based on RTTI (Run-Time Type Identification):
 - RTTI: allows programs to discover at runtime and use at runtime types that were not known at their compile time
 - Non-RTTI / Traditional approaches:
 - assume all types are known at compile time
 - Polymorphism in OO languages: is a particular case of very limited RTTI
- Sun: The Java Tutorials – The Reflection API
- <http://download.oracle.com/javase/tutorial/reflect/index.html>
- IBM developerWorks: Java programming dynamics, Part 2: Introducing reflection
- <http://www.ibm.com/developerworks/library/j-dyn0603/>

Kinds of tasks specific to Reflection

- **Inspection:** *analyzing objects and types to gather information about their definition and behavior.*
 - Find the run-time type information of an object
 - Find information about a type (supertypes, interfaces, members)
 - Dynamic type discovery
- **Manipulation:** *uses the information gained through inspection to change the structure/behavior:*
 - create new instances of new types discovered at runtime
 - dynamically invoke discovered methods
 - Late binding: the types and methods used by a program are not known at compile-time
 - The most one could imagine to do in a reflective language: restructure types and objects on the fly !

How is Reflection implemented

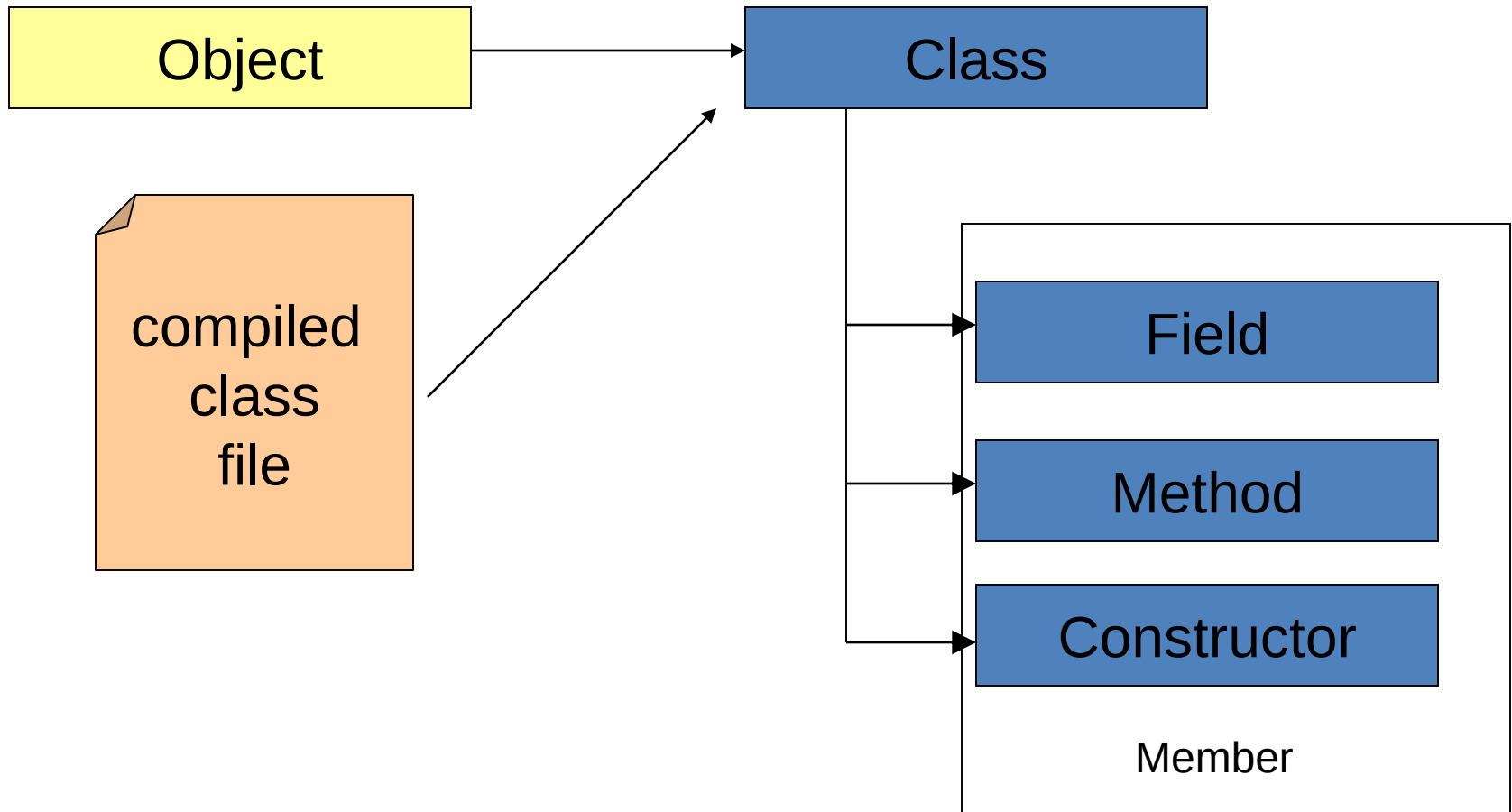
- Reflective capabilities need special support in language and compiler !
 - Java: `java.lang.reflection`
 - .NET: `System.Reflection`

Reflection case study:

Reflection in Java

- `java.lang.reflection`
- `Class java.lang.reflect.Class`
 - **It is the entry point for all of the Reflection API**
 - For each new class in a program a “Class” object is created.
 - Provides methods to examine the runtime properties of the object including its members and type information.
 - Provides the ability to create new objects of this type.

The Reflection Logical Hierarchy in Java



Retrieving a Class object

- **Object.getClass():** If **an instance of an object is available**, then the simplest way to get its [Class](#) is to invoke [Object.getClass\(\)](#).

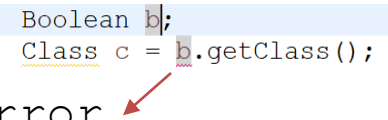
```
Class c = "foo".getClass();
```

- **.class:** If **the type is available but there is no instance** then it is possible to obtain a [Class](#) by appending ".class" to the name of the type. This is also the easiest way to obtain the [Class](#) for a primitive type.

```
boolean b;
```

```
Class c = b.getClass(); // compile-time error
```

```
Class c = boolean.class; // correct
```



```
Boolean b;  
Class c = b.getClass();
```

- **Class.forName():** If **the fully-qualified name** of a class is available, it is possible to get the corresponding [Class](#) using the static method [Class.forName\(\)](#)

```
Class cString = Class.forName("java.lang.String");
```

- Class是泛型类
 - `public final native Class<?> getClass()`
- `Class<Date> cls = Date.class;`
- 接口也有Class对象
 - `Class<Comparable> cls = Comparable.class;`
- 基本类型没有getClass方法，但也有对应的Class
 - `Class<Integer> intCls = int.class;`
 - `Class<Byte> byteCls = byte.class;`
- void是特殊的返回类型，也有Class
 - `Class<void> voidCls = void.class;`

- 对于数组，每种类型也有对应数组类型的Class对象，每个维度都有一个
 - `String[] strArr = new String[10];`
 - `int[][] twoDimArr = new int[3][2];`
 - `int[] oneDimArr = new int[12];`
 - `Class<? extends String[]> strArrCls = strArr.getClass();`
 - `Class<? extends int[][]>twoDimCls = twoDimCls.getClass();`
 - `Class<? extends int[]>oneDimCls = oneDimCls.getClass();`
- 枚举类型也有Class

```
Enum Size{
    SMALL, MEDIUM, BIG
}
Class<Size> cls = Size.class;
```

Inspecting a Class

- After we obtain a Class object `myClass`, we can:

- Get the class name

```
String s = myClass.getName() ;
```

- Get the class modifiers

```
int m = myClass.getModifiers() ;
```

```
bool isPublic = Modifier.isPublic(m) ;
```

```
bool isAbstract = Modifier.isAbstract(m) ;
```

```
bool isFinal = Modifier.isFinal(m) ;
```

- Test if it is an interface

```
bool isInterface = myClass.isInterface() ;
```

- Get the interfaces implemented by a class

```
Class [] itfs = myClass.getInterfaces() ;
```

- Get the superclass

```
Class super = myClass.getSuperClass() ;
```



```
String s = new String();
System.out.println(s.getClass().getSimpleName());
System.out.println(s.getClass().getCanonicalName());
System.out.println(s.getClass().getName());

Integer[] intArr = new Integer[] {};
System.out.println(intArr.getClass().getSimpleName());
System.out.println(intArr.getClass().getCanonicalName());
System.out.println(intArr.getClass().getName());

Integer[][] integers = new Integer[][] {};
System.out.println(integers.getClass().getSimpleName());
System.out.println(integers.getClass().getCanonicalName());
System.out.println(integers.getClass().getName());
```

<terminated> GETNAME [Java Applicatio

```
String
java.lang.String
java.lang.String
Integer[]
java.lang.Integer[]
[Ljava.lang.Integer;
Integer[][]
java.lang.Integer[][]
[[Ljava.lang.Integer;
```

- 1、getCanonicalName() 是获取所传类从java语言规范定义的格式输出。
- 2、getName() 是返回实体类型名称
- 3、getSimpleName() 返回从源代码中返回实例的名称。

Discovering Class members

- fields, methods, and constructors
- `java.lang.reflect.*` :
 - Member interface
 - Field class
 - Method class
 - Constructor class

Class Methods for Locating Members

Member	<u>Class</u> API	List of members?	Inherited members?	Private members?
<u>Field</u>	getDeclaredField()	no	no	yes
	getField()	no	yes	no
	getDeclaredFields()	yes	no	yes
	getFields()	yes	yes	no
<u>Method</u>	getDeclaredMethod()	no	no	yes
	getMethod()	no	yes	no
	getDeclaredMethods()	yes	no	yes
	getMethods()	yes	yes	no
<u>Constructor</u>	getDeclaredConstructor()	no	N/A ¹	yes
	getConstructor()	no	N/A ¹	no
	getDeclaredConstructors()	yes	N/A ¹	yes
	getConstructors()	yes	N/A ¹	no

Class Methods for locating Fields

- **getDeclaredField(String name):** Returns a **Field object** that reflects the specified declared field of the class or interface represented by this Class object. The **name** parameter is a String that specifies the simple name of the desired field.
- **getField(String name):** Returns a **Field object** that reflects the specified **public** member field of the class or interface represented by this Class object. The **name** parameter is a String specifying the simple name of the desired field. The field to be reflected is either declared in this class **or in a superinterface or superclass**.
- **getDeclaredFields():** Returns an array of **Field objects reflecting all the fields declared** by the class or interface represented by this Class object. **This includes public, protected, default (package) access, and private fields, but excludes inherited fields.** The elements in the array returned are not sorted and are not in any particular order
- **getFields():** Returns an array containing **Field objects reflecting all the accessible public fields** of the class or interface represented by this Class object. Specifically, if this Class object represents a class, this method returns the **public fields of this class and of all its superclasses**. If this Class object represents an interface, this method returns the fields of this interface and of all its superinterfaces.

Class Methods for locating Methods

- **getDeclaredMethod(String name):** Returns a **Method object** to the specified method, declared in this class
- **getMethod(String name):** Returns a **Method object** corresponding to the public specified method
- **getDeclaredMethods():** Returns an array of Method objects reflecting **all (public and private) the methods declared by the class** or interface represented by this Class object.
- **getMethods():** Returns an array containing Method objects reflecting **all the accessible public methods** of the class or interface represented by this Class object.

Class Methods for locating Constructors

- **getDeclaredConstructor (Class<?>... parameterTypes)**: Returns a **Constructor object** that reflects the specified constructor of the class or interface represented by this Class object. The parameterTypes parameter is an array of Class objects that identify the constructor's formal parameter types, in declared order.
- **getConstructor(Class<?>... parameterTypes)**: Returns a Constructor object that reflects the specified **public** constructor of the class represented by this Class object. The parameterTypes parameter is an array of Class objects that identify the constructor's formal parameter types, in declared order.
- **getDeclaredConstructors()**: Returns **an array of Constructor objects** reflecting all the constructors declared by the class represented by this Class object. **These are public, protected, default (package) access, and private constructors.** The elements in the array returned are not sorted and are not in any particular order.
- **getConstructors()**: Returns **an array containing Constructor objects reflecting all the accessible public constructors**

Working with Class members

- Members: fields, methods, and constructors
- For each member, the reflection API provides support to retrieve declaration and type information, and operations unique to the member (for example, setting the value of a field or invoking a method),
- `java.lang.reflect.*` :
 - “Member” interface
 - “Field” class: Fields have a **type** and a **value**. The [java.lang.reflect.Field](#) class provides methods for accessing type information and **setting and getting values** of a field on a given object.
 - “Method” class: Methods have **return values, parameters** and may throw **exceptions**. The [java.lang.reflect.Method](#) class provides methods for accessing type information for return type and parameters and **invoking** the method on a given object.
 - “Constructor” class: The Reflection APIs for constructors are defined in [java.lang.reflect.Constructor](#) and are similar to those for methods, with two major exceptions: first, constructors have no return values; second, the invocation of a constructor creates a new instance of an object for a given class.

Example

```
public class Btest
{
    public String aPublicString;
    private String aPrivateString;
    public Btest(String aString) {
        // ...
    }

    public Btest(String s1, String s2) {
        // ...
    }
    private void Op1(String s) {
        // ...
    }
}
protected String Op2(int x) {
    // ...
}
public void Op3() {
    // ...
}
}
```

```
public class Dtest extends Btest
{
    public int aPublicInt;
    private int aPrivateInt;
    public Dtest(int x)
    {
        // ...
    }

    private void OpD1(String s) {
        // ...
    }

    public String OpD2(int x){
        // ...
    }
}
```


Example: retrieving **public** fields

```
Class c = Class.forName("Dtest");
```

```
// get all public fields
```

```
Field[] publicFields = c.getFields();
```

```
for (int i = 0; i < publicFields.length; ++i) {
```

```
    String fieldName = publicFields[i].getName();
```

```
    Class typeClass = publicFields[i].getType();
```

```
    System.out.println("Field: " + fieldName + " of type " +
```

```
                        typeClass.getName());
```

```
}
```

```
Field: aPublicInt of type int
```

```
Field: aPublicString of type java.lang.String
```

Example: retrieving declared fields

```
Class c = Class.forName("Dtest");
```

```
// get all declared fields
```

```
Field[] publicFields = c.getDeclaredFields();
```

```
for (int i = 0; i < publicFields.length; ++i) {
```

```
    String fieldName = publicFields[i].getName();
```

```
    Class typeClass = publicFields[i].getType();
```

```
    System.out.println("Field: " + fieldName + " of type " +  
                                                                typeClass.getName());
```

```
}
```

```
Field: aPublicInt of type int  
Field: aPrivateInt of type int
```

Example: retrieving **public** constructors

```
// get all public constructors
```

```
Constructor[] ctors = c.getConstructors();
for (int i = 0; i < ctors.length; ++i) {
    System.out.print("Constructor (");
    Class[] params = ctors[i].getParameterTypes();
    for (int k = 0; k < params.length; ++k)
    {
        String paramType = params[k].getName();
        System.out.print(paramType + " ");
    }
    System.out.println(")");
}
```

```
Constructor (int )
```

Example: retrieving **public** methods

//get all public methods

```
Method[] ms = c.getMethods();
```

```
for (int i = 0; i < ms.length; ++i) {
```

```
    String mname = ms[i].getName();
```

```
    Class retType = ms[i].getReturnType();
```

```
    System.out.print("Method : " + mname + " returns " + retType.getName() + "
```

```
parameters :
```

```
Class[] param
```

```
for (int k = 0;
```

```
{
```

```
String p
```

```
System
```

```
}
```

```
System.out.p
```

```
}
```

```
Method : OpD2 returns java.lang.String parameters : ( int )
```

```
Method : Op3 returns void parameters : ( )
```

```
Method : wait returns void parameters : ( )
```

```
Method : wait returns void parameters : ( long int )
```

```
Method : wait returns void parameters : ( long )
```

```
Method : hashCode returns int parameters : ( )
```

```
Method : getClass returns java.lang.Class parameters : ( )
```

```
Method : equals returns boolean parameters : ( java.lang.Object )
```

```
Method : toString returns java.lang.String parameters : ( )
```

```
Method : notify returns void parameters : ( )
```

```
Method : notifyAll returns void parameters : ( )
```

Example: retrieving **declared** methods

//get all declared methods

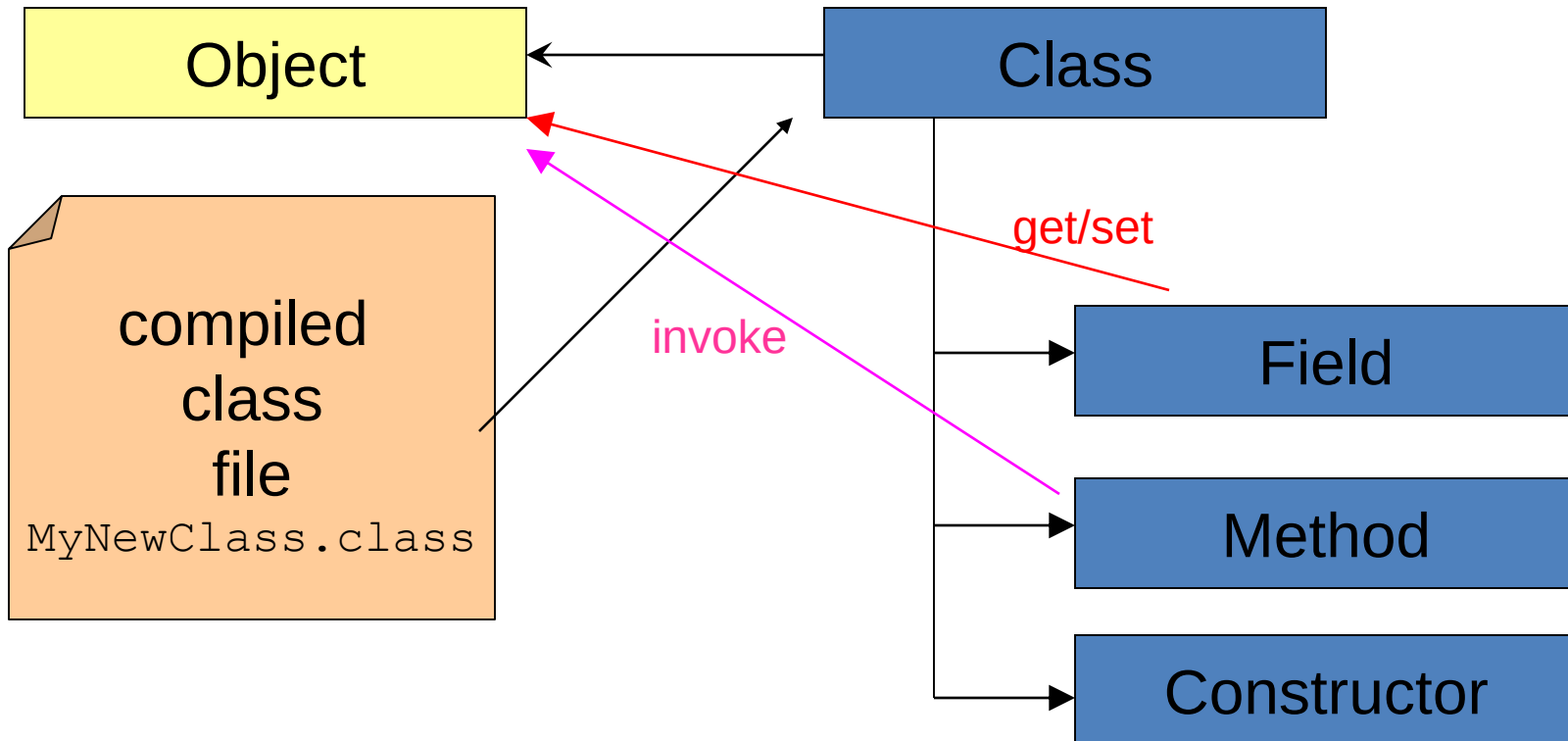
```
Method[] ms = c.getDeclaredMethods();
for (int i = 0; i < ms.length; ++i)      {
    String mname = ms[i].getName();
    Class retType = ms[i].getReturnType();
    System.out.print("Method : " + mname + " returns " + retType.getName() + "
parameters : ( ");
    Class[] params = ms[i].getParameterTypes();
    for (int k = 0; k < params.length; ++k)
        {
            String paramType = params[k].getName();
            System.out.print(paramType + " ");
        }
    System.out.println(" ");
}
```

```
Method : OpD1 returns void parameters : ( java.lang.String )
Method : OpD2 returns java.lang.String parameters : ( int )
```

Using Reflection for Program Manipulation

- Previous examples used Reflection for Introspection only
- Reflection is a powerful tool to:
 - **Creating new objects of a type** that was not known at compile time
 - **Accessing members** (accessing fields or invoking methods) that are not known at compile time

Using Reflection for Program Manipulation



Creating new objects

- Using Default Constructors
 - `java.lang.reflect.Class.newInstance()`

```
Class c = Class.forName("java.awt.Rectangle") ;  
Rectangle r = (Rectangle) c.newInstance() ;
```

- Using Constructors with Arguments
 - [java.lang.reflect.Constructor](#).`newInstance(Object... initargs)`

```
Class c = Class.forName("java.awt.Rectangle") ;  
Class[] intArgsClass = new Class[]{ int.class, int.class } ;  
Object[] intArgs = new Object[]{new Integer(12),new  
    Integer(24)} ;  
Constructor ctor = c.getConstructor(intArgsClass) ;  
Rectangle r = (Rectangle) ctor.newInstance(intArgs) ;
```


Accessing fields

- Getting Field Values

```
Rectangle r = new Rectangle(12,24) ;  
Class c = r.getClass() ;  
Field f = c.getField("height") ;  
Integer h = (Integer) f.get(r) ;
```

- Setting Field Values

```
Rectangle r = new Rectangle(12,24) ;  
Class c = r.getClass() ;  
Field f = c.getField("width") ;  
f.set(r,new Integer(30)) ;  
// equivalent with: r.width=30
```

Invoking methods

```
String s1 = "Hello " ;
String s2 = "World" ;
Class c = String.class ;
Class[] paramtypes = new Class[] { String.class } ;
Object[] args = new Object[] { s2 } ;
Method concatMethod =
    c.getMethod("concat",paramtypes) ;
String result = (String)
    concatMethod.invoke(s1,args) ;
// equivalent with result=s1.concat(s2) ;
```

Accessible Objects

- Can request that Field, Method, and Constructor objects be “accessible.”
 - Request granted if no security manager, or if the existing security manager allows it
- **Can invoke method or access field, even if inaccessible via privacy rules !**
- AccessibleObject Class: the Superclass of Field, Method, and Constructor
- `boolean isAccessible( )`
 - Gets the value of the accessible flag for this object
- `static void setAccessible( AccessibleObject[] array, boolean flag )`
 - Sets the accessible flag for an array of objects with a single security check
- `void setAccessible( boolean flag )`
 - Sets the accessible flag for this object to the indicated boolean value

Uses of Reflection

- Extensibility Features :
 - An application may make use of external, user-defined classes by creating instances of extensibility objects **using their fully-qualified names**.
- Class libraries that need to understand a type's definition
 - Typical example = Serialization
- Class Browsers and Visual Development Environments
 - **A class browser needs to be able to enumerate the members of classes**. Visual development environments can benefit from making use of type information available in reflection to aid the developer in writing correct code.
- Debuggers and Test Tools
 - **Debuggers need to be able to examine private members on classes**. Test harnesses can make use of reflection to systematically call a discoverable set APIs defined on a class, to insure a high level of code coverage in a test suite.

Drawbacks of Reflection

- If it is possible to perform an operation without using reflection, then it is preferable to **avoid using it**, because Reflection brings:
- **Performance Overhead**
 - Because reflection involves types that are dynamically resolved, certain Java virtual machine **optimizations can not be performed**. Consequently, reflective operations have slower performance than their non-reflective counterparts
- **Security Restrictions**
 - Reflection requires a runtime permission which may not be present when running under a security manager.
- **Exposure of Internals**
 - Since reflection allows code to perform operations that would be illegal in non-reflective code, such as accessing private fields and methods, **the use of reflection can result in unexpected side-effects**, which may render code dysfunctional and may destroy portability. Reflective code breaks abstractions and therefore may change behavior with upgrades of the platform.